# Thinking in Blender: Staged Executable Inverse Graphics with Vision-Language Models

GUANGZHAO HE*, Cornell University, USA
RUNDONG LUO*, Cornell University, USA
WEI-CHIU MA†, Cornell University, USA
HADAR AVERBUCH-ELOR†, Cornell University, USA


Fig. 1. From a single reference image (leftmost inset), SEIG reconstructs an editable Blender scene through a staged generator–verifier loop driven entirely by a pretrained VLM. As the output is a structured Blender program, our approach directly supports *novel-view synthesis*, *editing*, and *relighting* (right).

Inverse graphics is a longstanding and highly underconstrained problem that seeks to reconstruct images as editable 3D scenes which can be rendered, relit, and manipulated. In this work, we investigate whether pretrained vision-language models (VLMs) can perform executable inverse graphics directly from a single image by reconstructing a scene as an editable Blender program, without relying on specialized 2D or 3D foundation models, differentiable rendering, or multi-view supervision. We introduce **S**taged **E**xecutable **I**nverse **G**raphics (SEIG), an agentic framework that reconstructs a 3D scene from a single image by progressively refining scene factors including geometry, materials, composition, and lighting directly in executable Blender code space. We evaluate our framework across diverse scenes using a range of reconstruction metrics spanning pixel-level, perceptual, and semantic fidelity. Our experiments show that staged reconstruction substantially improves reconstruction fidelity, highlighting the importance of task decomposition for executable inverse graphics with general-purpose VLMs. Finally, we showcase various downstream applications enabled by the reconstructed editable Blender scenes.

## 1 Introduction

Creating 3D scenes is a complex and labor-intensive process that requires extensive expertise in specialized graphics software such as Blender. Professional artists typically construct scenes through a staged but highly iterative workflow: modeling or assembling geometry, assigning materials and textures, arranging scene composition, configuring lighting, and carefully adjusting camera parameters. Throughout this process, artists continuously inspect intermediate results and iteratively refine individual scene factors until the final scene matches the desired visual target. Replicating this capability

*Denotes equal contribution.
†Denotes equal advising.

Authors' Contact Information: Guangzhao He, Cornell University, USA, gh466@cornell.edu; Rundong Luo, Cornell University, USA, rl897@cornell.edu; Wei-Chiu Ma, Cornell University, USA, wm347@cornell.edu; Hadar Averbuch-Elor, Cornell University, USA, hadarelor@cornell.edu.

automatically from images has long been a central goal of inverse graphics [Barrow et al. 1978; Marschner 1998; Roberts 1963].

Recent advances in vision-language models (VLMs) have demonstrated remarkable capabilities in visual reasoning, instruction following, and code generation [Hu et al. 2024; Liu et al. 2023a; Yin et al. 2026], suggesting that these models may encode rich latent knowledge about 3D scenes and their underlying structure. In this work, we investigate whether pretrained VLMs can perform executable inverse graphics directly from a single image. Specifically, we ask whether VLMs can reconstruct a scene as an editable Blender program by recovering its underlying factors—including geometry, materials, composition and lighting—without relying on specialized 2D or 3D foundation models, differentiable rendering pipelines, or large-scale multi-view supervision.

Our key observation is that, while pretrained VLMs struggle to reconstruct all scene factors simultaneously, their executable inverse graphics capabilities can be unlocked by decomposing reconstruction into sequential, semantically meaningful stages that mirror the iterative workflow used by professional 3D artists. Building on this observation, we introduce SEIG, a **S**taged **E**xecutable **I**nverse **G**raphics framework built on top of a pretrained vision-language model. Starting from a single input image, SEIG first initializes a coarse scene scaffold composed of simple geometric primitives and approximate object layouts, and then progressively refines this representation through sequential stages that recover geometry, materials, object composition and lighting directly in executable Blender code space. Each stage is paired with a verifier module that renders and evaluates the current scene state before guiding subsequent refinements. By decomposing inverse graphics into sequential executable refinement stages, our framework reduces the complexity of the overall reconstruction problem while maintaining a fully editable and physically grounded scene representation throughout the reconstruction process.

We demonstrate our approach across both synthetic and in-the-wild scenes, comparing against monolithic executable inverse graphics baselines with and without specialist 2D and 3D foundation models. Our experiments show that staged reconstruction substantially improves reconstruction fidelity, suggesting that task decomposition may play a more critical role than the richness of the external toolkit used by the pipeline and that pretrained vision-language models encode surprisingly rich latent priors about 3D structure, appearance, and scene composition. Finally, we demonstrate the applicability of our approach to downstream graphics tasks including relighting, scene editing and physics simulation, enabled directly through the reconstructed editable Blender scene representation.

## 2 Related Work

**Inverse Graphics**. Recovering structured 3D scenes from 2D images has long been a central goal in computer vision and graphics, dating back to early formulations of inverse graphics such as Roberts' "Blocks World" [Roberts 1963]. Early works in inverse graphics primarily focused on the inverse rendering problem, seeking to recover scene geometry, illumination, and reflectance from images through analysis-by-synthesis formulations [Marschner 1998; Ramamoorthi and Hanrahan 2001], intrinsic decomposition [Barrow et al. 1978] and shape-from-shading [Horn 1970]. More recent approaches extend these ideas to recovering geometry and reflectance from sparse views or single images using neural networks [Bi et al. 2020; Dong et al. 2014; Li et al. 2018; Nam et al. 2018], while several works further explore structured scene decomposition through differentiable rendering and primitive-based representations [Monnier et al. 2023; Sharma et al. 2018].

In recent years, inverse graphics research has increasingly shifted toward neural scene representations such as NeRF [Mildenhall et al. 2020] and 3D Gaussian Splatting [Kerbl et al. 2023]. While these neural representations are highly effective for scene reconstruction, they typically encode geometry, materials, and lighting in latent neural representations that are not directly editable as structured graphics programs. Several works seek partial disentanglement within this paradigm by factoring shape and reflectance [Zhang et al. 2021a,b], separating geometry, BRDFs, and illumination [Jin et al. 2024; Liang et al. 2024; Munkberg et al. 2022], or introducing object-level scene decomposition [Benaim et al. 2024; Luo et al. 2025; Wu et al. 2024; Yang et al. 2021]. Despite this progress, these methods still do not directly recover executable scene programs.

**Vision-Language Models for 3D Reasoning.** Vision-language models (VLMs) demonstrate strong capabilities in visual understanding, instruction following, and code generation [Gemini Team 2023; Liu et al. 2023b; OpenAI 2023]. Beyond semantic reasoning, several works show that these models also encode non-trivial spatial and geometric understanding from 2D observations. Prior studies demonstrate that, with appropriate prompting, VLMs can perform coarse 3D grounding, spatial reasoning, and geometric estimation from images [Liu et al. 2023a], while benchmarks such as SpatialVLM [Chen et al. 2024] systematically evaluate these capabilities across diverse spatial tasks. At the same time, recent evaluations reveal that current VLMs remain significantly stronger at semantic reasoning than at precise geometric prediction, particularly in tasks requiring accurate spatial localization or metric 3D understanding [Kulits et al. 2024]. These observations motivate our investigation into whether pretrained VLMs can nevertheless support executable inverse graphics from a single input image through staged reconstruction and iterative visual verification.

**Executable Scene Generation with Vision-Language Models.** Recent works explore using vision-language models to generate and manipulate 3D content through executable scene representations. SceneCraft [Hu et al. 2024] and LL3M [Lu et al. 2025] investigate generating executable 3D scenes as Blender programs from text instructions. MeshCoder [Dai et al. 2025] instead targets code-based mesh generation from point clouds, while BrickGPT [Pun et al. 2025] explores compositional 3D generation through structured primitive-based representations. Articulate-Anything [Le et al. 2024] and VDAWorld [O'Mahony et al. 2025] further extend these ideas to generating articulated assets and simulation-ready environments from multimodal inputs. BlenderGym [Gu et al. 2025] and IR3D-Bench [Liu et al. 2025] focus on evaluating VLM-driven 3D editing and reconstruction systems, highlighting geometric precision and spatial consistency as major limitations of current models.

Closest to our work, VIGA [Yin et al. 2026] formulates vision-as-inverse-graphics as an iterative write–render–compare–revise

loop, enabling executable 3D scene reconstruction from multimodal inputs, including the single image setting addressed in our work. As illustrated in our experiments, however, treating reconstruction as a single monolithic optimization problem remains highly challenging for current VLMs, which struggle to jointly reason about geometry, materials, composition and lighting, within a single entangled generation process. In contrast, our framework explicitly decomposes reconstruction into sequential generator–verifier stages that independently recover individual scene factors. Furthermore, unlike prior work, our approach operates entirely using a single pretrained VLM without relying on specialized 2D or 3D foundation models, such as SAM [Kirillov et al. 2023] and SAM-3D [Meta AI Research 2025]. This design allows us to more directly investigate the extent to which pretrained VLMs themselves encode the geometric, spatial, and compositional priors necessary for executable inverse graphics.

## 3 Method

In this section, we introduce SEIG: a framework that takes a reference image as input, reconstructs the underlying scene by decomposing the problem into stages, and produces executable Blender code whose render matches the input image. We use Blender as the reconstruction engine since it provides a unified Python interface for scene editing and image rendering, allowing the agent to modify the scene and observe it via simple API calls. Rather than requiring hours of manual construction and refinement by an experienced artist, SEIG instead delegates this workflow to a vision-language model (VLM) that interprets the input image, reasons about scene structure, and interacts with Blender through tool calls.

However, directly prompting a VLM to generate or refine a complete Blender scene places a heavy burden on the model, requiring it to jointly infer geometry, materials, composition, and lighting parameters while producing executable code. Prior VLM-based inverse graphics agents [Gu et al. 2025; Yin et al. 2026] demonstrate the promise of this direction, but still leave many coupled scene factors to be solved at once. In this work, we address this by decomposing inverse graphics into independently scoped, verifiable subproblems. Central to our approach is a multi-stage, additive pipeline in which the scene is constructed and refined autoregressively. Each stage is disentangled to depend only on earlier ones (e.g., geometry before material refinement, material refinement before lighting), so the framework can commit to each subproblem independently before proceeding (Sec. 3.1). Within each stage, a generator-verifier loop drives iterative refinement: the generator writes Blender code across multiple tool-use rounds, then passes the rendered result to the verifier, which decides whether to request a revision or advance to the next stage (Sec. 3.2). An overview is shown in Fig. 2.

### 3.1 Staged Scene Construction

Inverse graphics requires solving a set of tightly coupled subtasks, including reconstructing object geometry and material appearance, predicting the scene layout, and recovering environment lighting. With each of these subtasks being challenging even for human artists or specialist models [Gao et al. 2024; Jin et al. 2024; Verbin et al. 2022; Wang et al. 2025; Zhang et al. 2024], asking a VLM agent to directly generate code for the entire scene is therefore a highly underconstrained problem. While prior works [Gu et al. 2025; Yin et al. 2026] attempt to generate and refine the entire scene at once, we argue that optimizing all such factors jointly creates a large search space in which errors in one factor can obscure corrections to another. To this end, we propose a greedy, staged approach, in which the problem is decomposed into verifiable substages, enabling more grounded reasoning, and allowing the VLM to additively construct the scene through disentangled steps.

More specifically, we follow the conventional workflow used by human artists when creating renders in Blender: initializing the scene, modeling individual objects, drawing texture maps and assigning materials, composing the objects into a scene, and finally adding environment lighting. We formulate each stage as an agentic function that depends only on the outputs of previous stages while maintaining its own stage-specific context for reasoning.

**Scene decomposition.** Given a reference image, we first prompt the VLM to decompose the scene into a hierarchical scene graph that covers all visible objects. The graph contains a scene root for the global environment and object nodes for physical objects or object parts. Each node stores a visual description, approximate geometry, material appearance, spatial relations to its parent and nearby nodes, and a Blender reconstruction strategy. The VLM then recursively refines the graph until each leaf node corresponds to an atomic component that can be approximated with Blender primitives (e.g., spheres, cubes or cones). This representation encourages full scene coverage and provides later stages with referable object names for localized refinement.

**Scene initialization.** Given the scene graph, the scene initialization stage instantiates a coarse Blender scaffold containing all components in the graph. The VLM first produces an execution plan from the scene-graph attributes, then generates Blender code that creates each leaf node with an initial geometry and material approximation according to the plan. This scaffold, built only from textual descriptions, is not intended to match the reference image precisely, but to ensure that every decomposed component exists in the scene and is assigned a stable Blender object name across stages, providing a consistent reference throughout. During this process, it also initializes the lighting and camera coarsely to ensure that all scene components are clearly visible without cropping or over-exposure.

Since scene initialization determines the decomposition and coarse scaffold used by all later stages, failure to cover all objects or bad association can be difficult to recover from local refinement alone. We therefore use a rollout sampling strategy during initialization: we sample multiple independent scene graphs and coarse Blender scaffolds, then apply a rollout selector to select the candidate with the most complete object coverage and most plausible structure. The selected rollout is then passed to the remaining stages, where iterative refinement optimizes different factors in sequence.

**Geometry stage.** After initialization produces a coarse scaffold, in which each scene graph leaf node is instantiated as a named Blender object, the geometry stage refines the shape of each individual object. More specifically, a VLM is asked to refine each object's shape and physical structure through three classes of edits: (1) local shape edits, such as adjusting meshes and curves; (2) geometric transforms, such as scaling, rotating, and aligning existing objects parts; and (3)

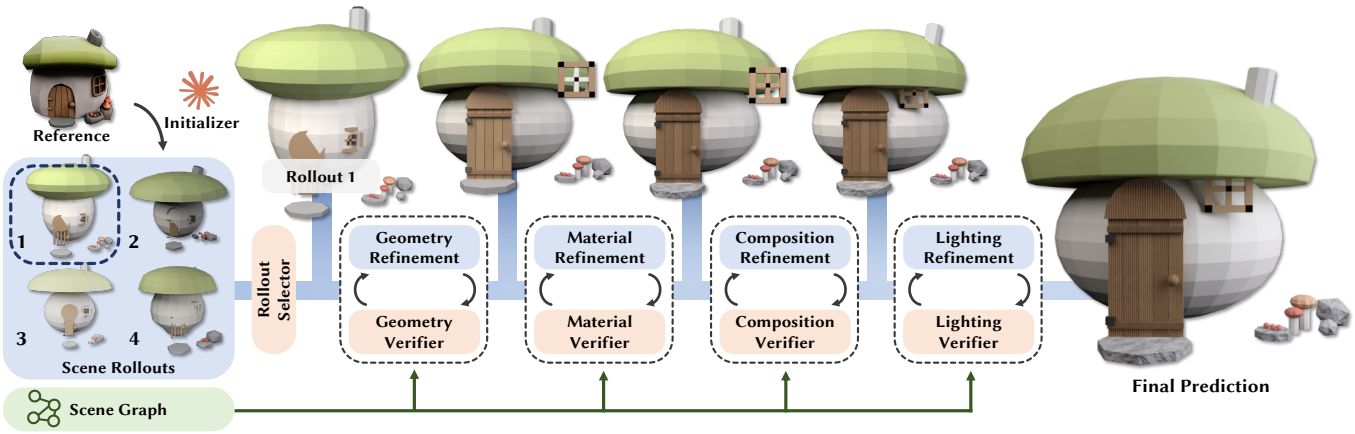


Fig. 2. **Method overview.** Our agentic reconstruction pipeline decomposes inverse graphics into four sequential stages of Blender code generation, each consisting of a *generator* step followed by a *verifier* step. An initialization stage samples four coarse scene initializations and labels a scene graph for all objects and parts, which is then passed to all subsequent stages for object- and part-centric refinement. The scene graph records each object with attributes such as geometry, material, and spatial relations. The final scene is produced by executing the refined Blender code, reconstructing the reference image.

structural edits, such as adding missing parts or organizing object's internal hierarchies. To support object-centric refinement, we provide the VLM with interaction tools for inspecting and editing the scene. The agent can call them to render the scene from alternative viewpoints, isolate individual objects for focused inspection, and revert unsuccessful edits when visual feedback indicates a regression. The result from this stage is a geometrically refined scene with proper object identities, providing the structural foundation needed for subsequent material refinement and scene-level composition.

**Material stage.** After object geometry is refined, the material stage completes the object-wise reconstruction by matching each object's material and surface appearance with the reference image. While the initialization may provide coarse, often single-color textures, the material stage replaces these placeholders with more detailed Blender PBR materials. For each object in the scene graph, the agent initializes material slots and UV maps when needed, then creates procedural or image textures through Blender shader nodes. The agent edits surface properties such as base color, roughness, specular, metallic, alpha, and bump or normal maps, capturing material identity and local texture detail. To prevent material refinement from altering earlier stage results, the model executes Blender code through a material-only tool that permits only material-related edits.

**Composition stage.** After object-level geometry and material refinement, the composition stage arranges the finalized objects to match the reference image at the scene layout level. Unlike the previous object-centric stages, this stage compares the target-view render against the reference and adjusts object transforms to match their relative scale, position, rotation, contact, and overall spatial organization. During this process, the VLM agent may adjust the target-view camera when necessary to compare the scene with the reference, and may freely use temporary arbitrary-view renders to judge layout from other viewpoints. However, the agent is not allowed to edit object geometry or materials.

**Lighting stage.** After geometry, material, and composition refinement, the lighting stage serves as the final scene-level adjustment, matching the rendered appearance of the accumulated Blender scene to the reference image by optimizing lighting parameters while keeping object shape, appearance, layout, and camera fixed. The lighting stage agent compares the current render with the reference image to infer lighting cues such as light direction and height, shadow direction and softness, color temperature, exposure, and contrast. The agent can adjust both the physical lighting controls, including light type, position, direction, energy, color, size, softness, and ambient lighting, as well as render settings such as exposure and color management. Since lighting parameters are sensitive to small changes, the agent is instructed to make conservative edits and revert changes that make the render too dark or overexposed.

## 3.2 Intra-Stage Generator-Verifier Refinement

While our staged pipeline decomposes inverse graphics into simpler subproblems, each stage still requires iterative refinement since a single code-generation pass is insufficient for matching the reference image. Therefore, we adopt a multi-round generator-verifier loop per stage, where the generator calls tools to inspect the current Blender scene, writes stage-specific code, executes the edit, and renders the updated result. After each generator attempt, a verifier compares the rendered image against the reference and inspects the scene through its own set of tools to identify remaining stage-specific mismatches. Crucially, each verifier is scoped to its corresponding stage: it judges only the active factor (*e.g.*, object presence for initialization), while ignoring errors assigned to other stages.

Free-form verifier critiques can be noisy across attempts, giving the generator inconsistent targets and preventing convergence. We therefore require the verifier to return an explicit approval checklist: a concrete, actionable todo list of visual discrepancies that is injected into the generator context for the next attempt, and once the generator attempt satisfies these conditions, the verifier must approve it to move on to the next stage. To prevent the refinement effectiveness from degrading over time due to accumulated context, we impose a stage-specific maximum round budget. If each generator-verifier

loop reaches its budget without satisfying the checklist, the verifier then must select the best attempt to move forward to the next stage. We set the round budget according to the complexity of each stage. In practice, we allow five rounds for geometry refinement, three rounds each for material and composition refinement, and two rounds for lighting refinement.

## 4 Experiments

In this section, we describe our main experimental results. Specifically, we seek to answer the following questions:

- Can a staged executable inverse graphics framework, built entirely on top of an off-the-shelf VLM, produce structured Blender reconstructions that faithfully recover geometry, appearance, layout, and lighting from a single image?
- How much of the reconstruction quality stems from the staged reconstruction framework itself, particularly when compared against existing executable inverse graphics systems both with and without specialized external tools?
- Are the resulting scenes genuinely useful as graphics assets, supporting standard downstream operations such as novel-view synthesis, relighting, and object editing without any further training?

In what follows, we first describe our experimental setup (Section 4.1). We then present quantitative and qualitative comparisons and results (Sections 4.2 and 4.3, respectively). Finally, we showcase the downstream applications enabled by the reconstructed scene representation (Section 4.4) and discuss limitations (Section 4.5). In the supplementary material, we provide full prompts used by our systems along with chat history for a sample input image.

### 4.1 Setup

*Implementation Details.* We use Claude Opus 4.7 [Anthropic 2025], accessed through the Anthropic API, as the base VLM for both the generator and the verifier at every stage of our pipeline. The same model is used throughout all experiments without any fine-tuning, prompt-tuning, or task-specific supervision, so any observed difference in reconstruction quality between methods can be attributed to harness design rather than the underlying model.

*Baselines and Metrics.* We compare our framework with VIGA [Yin et al. 2026], the closest monolithic agentic baseline for executable inverse graphics, in two configurations. VIGA$^{\text{full}}$ is the original VIGA pipeline, which uses SAM [Kirillov et al. 2023] and SAM-3D [Meta AI Research 2025] to pre-segment and pre-reconstruct individual objects before invoking its agentic write–render–compare–revise loop. VIGA$^{\text{VLM-only}}$ is an ablation of VIGA that disables these specialist 2D and 3D foundation models, leaving only the VLM-driven agentic loop. Reporting both isolates how much of VIGA's performance comes from its specialist toolchain versus its VLM agent, and makes the comparison with our pipeline—which also relies on the VLM alone—an apples-to-apples test of harness design. For a fair comparison, all methods share the same backbone VLM.

For quantitative evaluation, we curate a set of images from the NeRF synthetic dataset [Mildenhall et al. 2020] and VoxHammer [Li et al. 2026]. Five images are rendered from 7 of the 8 NeRF scenes; the *materials* scene containing specular reflections from metallic spheres is excluded from our evaluation. Likewise, we gather 13 object-centric scenes from [Li et al. 2026]. We report six metrics between each reconstructed rendering and its reference image: PSNR and SSIM at the pixel level; LPIPS and DreamSim [Fu et al. 2023] as learned perceptual scores; and two semantic similarities, DINO [Oquab et al. 2024] (cosine similarity between [CLS]-token features of a DINOv2 ViT-L/14 encoder) and CLIP [Radford et al. 2021] (cosine similarity between image embeddings from a CLIP ViT-B/32 encoder). When reference meshes are available, we avoid conflating reconstruction quality with the agent's camera estimate by registering each reconstructed mesh to the reference by running both Neural Deformation Pyramid (NDP) [Li and Harada 2022] and ICP and keeping whichever yields the smaller Chamfer distance, then rendering the aligned scene from the reference camera and computing the six metrics on that rendering. Additionally, we collect a set of images to stress-test the framework on in-the-wild scenarios.

| Method | PSNR ↑ | SSIM ↑ | LPIPS ↓ | DreamSim ↓ | DINO ↑ | CLIP ↑ |
|---|---|---|---|---|---|---|
| VIGA$^{\text{VLM-only}}$ | 12.33 | **0.7122** | 0.3506 | 0.3693 | 0.6221 | 0.8451 |
| VIGA$^{\text{full}}$ | 11.18 | 0.6647 | 0.3944 | 0.3624 | 0.5545 | 0.7986 |
| **Ours** | **13.58** | 0.6881 | **0.3493** | **0.3021** | **0.7188** | **0.8830** |

Table 1. **Quantitative comparison on NeRF synthetic scenes.** PSNR, SSIM, LPIPS, DreamSim, DINO, and CLIP between each method's reconstructed rendering and the reference image.

| Method | PSNR ↑ | SSIM ↑ | LPIPS ↓ | DreamSim ↓ | DINO ↑ | CLIP ↑ |
|---|---|---|---|---|---|---|
| VIGA$^{\text{VLM-only}}$ | 11.52 | **0.6776** | 0.3931 | 0.3847 | 0.5606 | 0.8366 |
| VIGA$^{\text{full}}$ | 12.48 | 0.6743 | 0.4466 | 0.4441 | 0.4832 | 0.7883 |
| **Ours** | **12.65** | 0.6737 | **0.3823** | **0.3433** | **0.6293** | **0.8446** |

Table 2. **Quantitative comparison on Edit3D.** We report the same metrics as in Tab. 1 over scenes gathered from [Li et al. 2026].

### 4.2 Quantitative Results

Tab. 1 and Tab. 2 report the quantitative comparison on the NeRF synthetic and Edit3D sets. SEIG achieves the best score on five out of six metrics on both the NeRF synthetic and Edit3D scenes. Despite using no specialist 2D or 3D foundation models, SEIG outperforms VIGA$^{\text{full}}$, indicating that the gains come from harness design rather than tool access; that it also outperforms VIGA$^{\text{VLM-only}}$ verifies the contribution of our per-stage decomposition. This is consistent with the finding from BlenderGym [Gu et al. 2025] and IR3D-Bench [Liu et al. 2025] that visual precision, not tool orchestration, is the dominant bottleneck in current agentic 3D pipelines.

### 4.3 Qualitative Results

Fig. 8 shows representative reconstructions produced by our pipeline. Across these scenes, our framework produces structured Blender outputs that match the reference images in geometry, surface appearance, and composition, demonstrating that careful harness design alone, without any task-specific training, is sufficient to produce high-quality inverse graphics from an off-the-shelf VLM. Fig. 3 compares these reconstructions with both VIGA variants; our method produces more accurate reconstructions than either configuration across geometry, material, and composition on most cases, while VIGA$^{\text{VLM-only}}$ loses smaller scene elements and VIGA$^{\text{full}}$ fragments


Fig. 3. **Qualitative comparison across methods.** Each block shows a different reference image (leftmost column) together with the reconstructions produced by VIGA$^{\text{VLM-only}}$, VIGA$^{\text{full}}$, and our pipeline. Within each method's column, the large rendering is taken from the reference viewpoint, and the two smaller show the same reconstructed scene rendered from alternate viewpoints, revealing the underlying 3D structure. Across these scenes, our pipeline reproduces the global geometry, materials, and object composition of each reference despite using no specialist 3D foundation models; VIGA$^{\text{VLM-only}}$ recovers a plausible silhouette but loses smaller scene elements and surface detail, while VIGA$^{\text{full}}$ often produces fragmented, mis-colored meshes that fail to assemble into a coherent scene since the VLM agent may overwrite the texture of the 3D objects generated by SAM-3D.

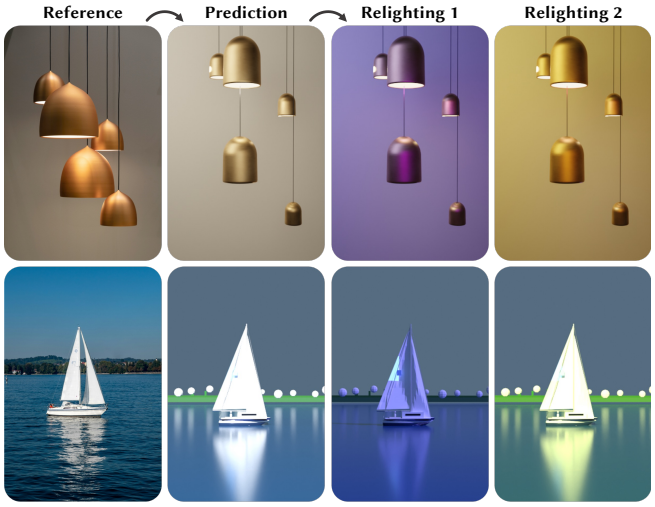


Fig. 4. **Relighting.** Two reconstructed scenes (top: pendant lamps; bottom: sailboat) re-rendered under two new lighting configurations. Since lights are separately stored in Blender, new illumination can be applied by adding or reconfiguring light sources without re-running any part of the pipeline.

into disjoint, mis-colored meshes, as the underlying VLM agent often overwrites the texture of the generated 3D objects.

As can be observed in the fourth example of Fig. 3 (the humanoid character), even a strong single-image 3D generator such as SAM-3D may lead to characteristic single-view lifting artifacts. In this case, VIGA$^{\text{full}}$ exhibits the well-known *Janus* failure mode, where frontal facial features are duplicated onto the back of the character's head due to ambiguity in the unseen object regions. In contrast, VIGA$^{\text{VLM-only}}$ and our pipeline avoid this failure mode by reconstructing the figure compositionally from primitives rather than lifting a single-view mesh prior. The bread-basket scene in the bottom of Fig. 3 illustrates the inherently underdetermined nature of single-view reconstruction: the reference's contents are mostly occluded, and although the ground truth is a basket of bread sticks, our pipeline instead produces rounded loaves—an interpretation equally consistent with the visible silhouette and rendered as a perceptually coherent scene. Both VIGA variants, by contrast, fail to recover even a coherent basket structure on the same input, indicating that their failure mode here is not view ambiguity but a lack of compositional discipline in the monolithic agentic loop.

Beyond the final reconstructions, Fig. 7 traces the pipeline's intermediate outputs through each stage on two example scenes, demonstrating the importance of our staged approach. Starting from a coarse primitive-based scaffold, the four stages progressively refine the scene: geometry refinement sharpens individual object shapes, material assignment adds surface details, composition places the objects in their reference layout, and lighting configures the illumination. The final image is then rendered from a VLM-determined camera. Because each stage commits its output before the next begins, every intermediate scene itself is a coherent, editable Blender program—a property that distinguishes staged reconstruction from monolithic agentic pipelines and makes the pipeline's decisions inspectable and selectively reusable at any stage boundary.

### 4.4 Applications

A key advantage of producing an editable Blender file rather than an entangled latent representation is allowing immediate graphics operations without retraining or post-processing. In the following, we showcase examples on performing *relighting*, *object editing*, and *physics simulation* on our reconstructed scenes.

**Relighting.** Because geometry, materials, and light sources are committed as separate stage outputs (see the *Lighting* column of Fig. 7), lights can be added, removed, or reconfigured and the scene re-rendered under arbitrary new illumination without touching the rest of the recovered scene. The two examples in Fig. 4 show our reconstructions re-rendered under two synthetic lighting configurations each, producing changes in dominant color, direction, and intensity while preserving the recovered geometry and materials.

**Object editing.** The same structure makes per-object edits trivial: because each object is built independently in the Geometry and Material stages and only later assembled by the Composition stage, any node of the scene graph can be selected, moved, duplicated, retextured, or replaced directly in Blender. Fig. 5 shows four representative operations on two recovered scenes—*part duplication* and *texture editing* on an aircraft, and *shape manipulation* and *object composition* on a castle—each obtained by a small manual edit on the existing scene file. Fig. 7 additionally shows *object rearrangement* on a recovered tabletop. None of these operations is straightforwardly available on the entangled latent representations produced by monolithic neural reconstruction methods.

**Physics Simulation.** Because the reconstructed scene is a structured collection of separately addressable meshes in Blender, it is also a valid input to Blender's built-in physics engine. Fig. 6 shows two example scenarios run directly on our outputs: shaking the tabletop in the top row triggers rigid-body interactions among the recovered mugs and saucers, while dropping a ball onto the recovered couch in the bottom row exercises soft-body deformation of the cushion. Beyond attaching the appropriate Blender physics modifier (*rigid body* or *soft body*) to the relevant scene-graph nodes, neither scene requires remeshing, watertighting, or any other geometry repair—a direct consequence of the pipeline producing object-decomposed meshes rather than a single fused implicit representation, which would otherwise need to be converted into discrete physical entities before any simulation could be defined.

### 4.5 Limitations

While our staged formulation enables more reliable executable inverse graphics reconstruction, errors introduced in early stages may propagate throughout the pipeline, leading to local minima from which later stages cannot easily recover. For example, inaccurate geometric reconstruction may constrain subsequent material, lighting, or compositional reasoning. One possible direction for mitigating this limitation would be to introduce additional global refinement passes that revisit and jointly optimize earlier scene factors after later reconstruction stages. However, such multi-round optimization would come at the expense of substantially increased computational cost and inference time. Another notable limitation is the computational expense of repeated inference calls across multiple generator–verifier stages, resulting in substantially higher runtime and API cost compared to single-pass generation pipelines.

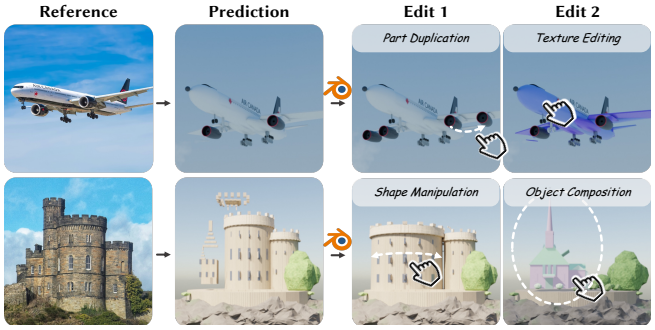


Fig. 5. **Object editing.** Two reconstructed scenes (top: aircraft; bottom: castle), each shown alongside two example edits performed directly in Blender on the recovered scene graph: *part duplication* and *texture editing* for the aircraft; *shape manipulation* and *object composition* for the castle.

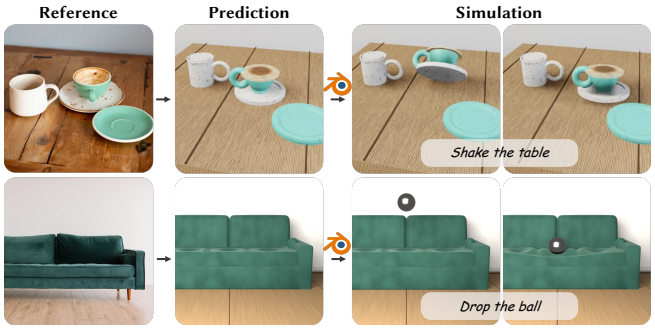


Fig. 6. **Physics simulation.** Two reconstructed scenes used as input to Blender's built-in physics engine. Top: rigid-body dynamics—the recovered mugs and saucers slide and rattle when the table is given an external acceleration (*shake the table*). Bottom: soft-body dynamics—a ball is dropped onto the recovered cushion, which deforms accordingly (*drop the ball*). Both simulations run directly on the reconstructed scene graph in Blender.

## 5 Conclusion

We introduced a staged executable inverse graphics framework that reconstructs editable Blender scenes directly from a single image using only a pretrained vision-language model, without task-specific training, specialized foundation models, or differentiable rendering. By decomposing reconstruction into a sequence of individually verifiable subproblems, each closed by its own generator–verifier stage, our framework enables the model to progressively recover geometry, materials, composition and lighting, while avoiding the entangled-output bottleneck that limits monolithic reconstruction pipelines. Future work could extend these ideas beyond static single-image reconstruction toward more challenging settings such as multi-image scene reconstruction, dynamic environments, physically grounded simulation, and long-horizon interactive editing. More broadly, our results suggest that staged executable scene representations provide a promising pathway for transforming increasingly capable general-purpose VLMs into controllable 3D content creation systems.

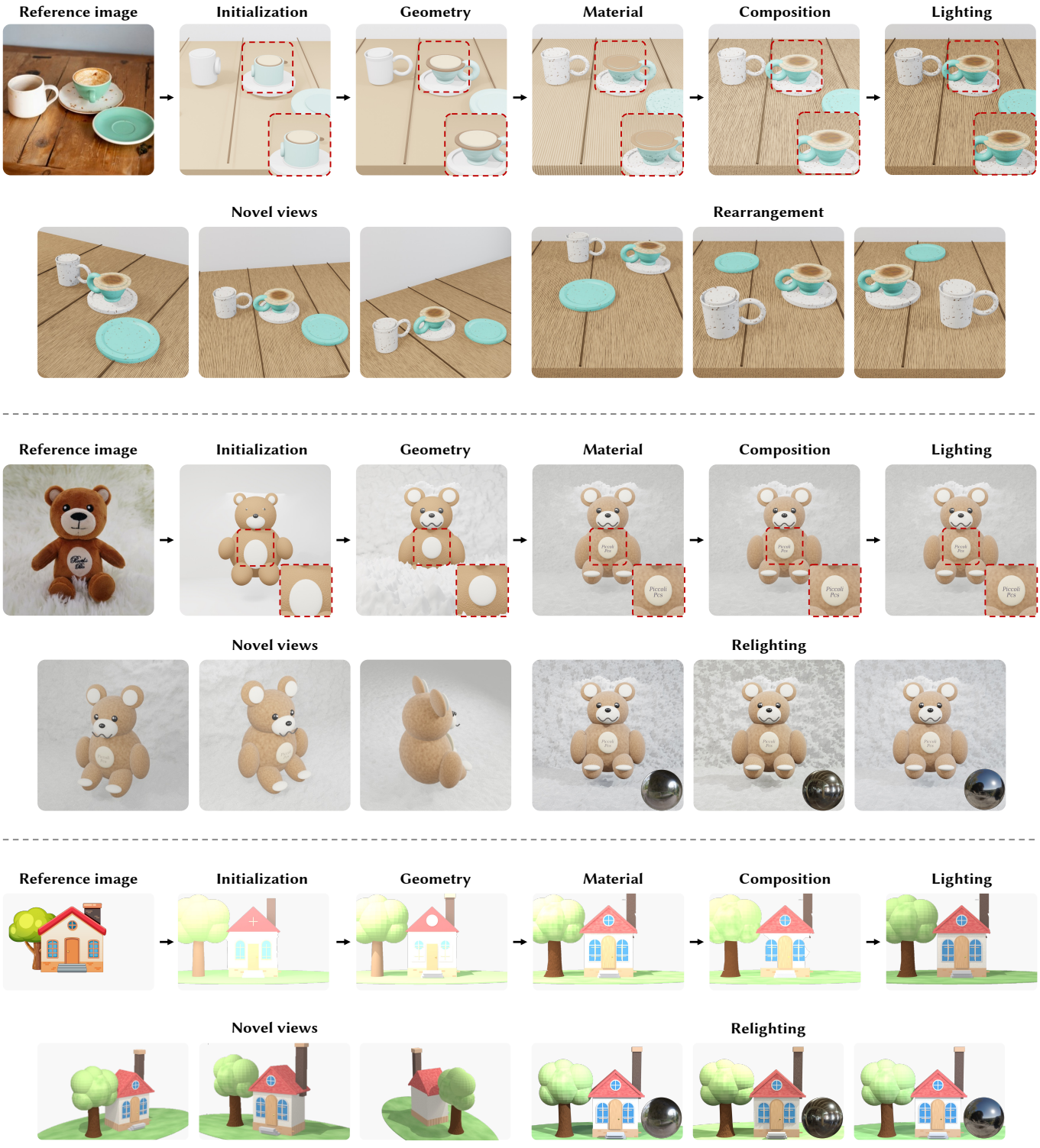


Fig. 7. **Intermediate outputs across pipeline stages.** Two examples showing the rendered scene through our pipeline: starting from a coarse initial scaffold (*Initialization*), through the four stages—*Geometry*, *Material*, *Composition*, and *Lighting*—each closed by its own generator–verifier loop, and the final image rendered from a VLM-determined camera (*Camera-adjustment*, rightmost column). Each stage commits its output before the next begins, so every intermediate scene is itself a coherent, editable Blender program. This staged structure also underlies the downstream applications in Sec. 4.4: materials, lights, and individual objects are exposed as separate, named stage outputs, so each can be modified directly without rerunning earlier stages—enabling the relighting, object editing, and physics simulation results that follow.

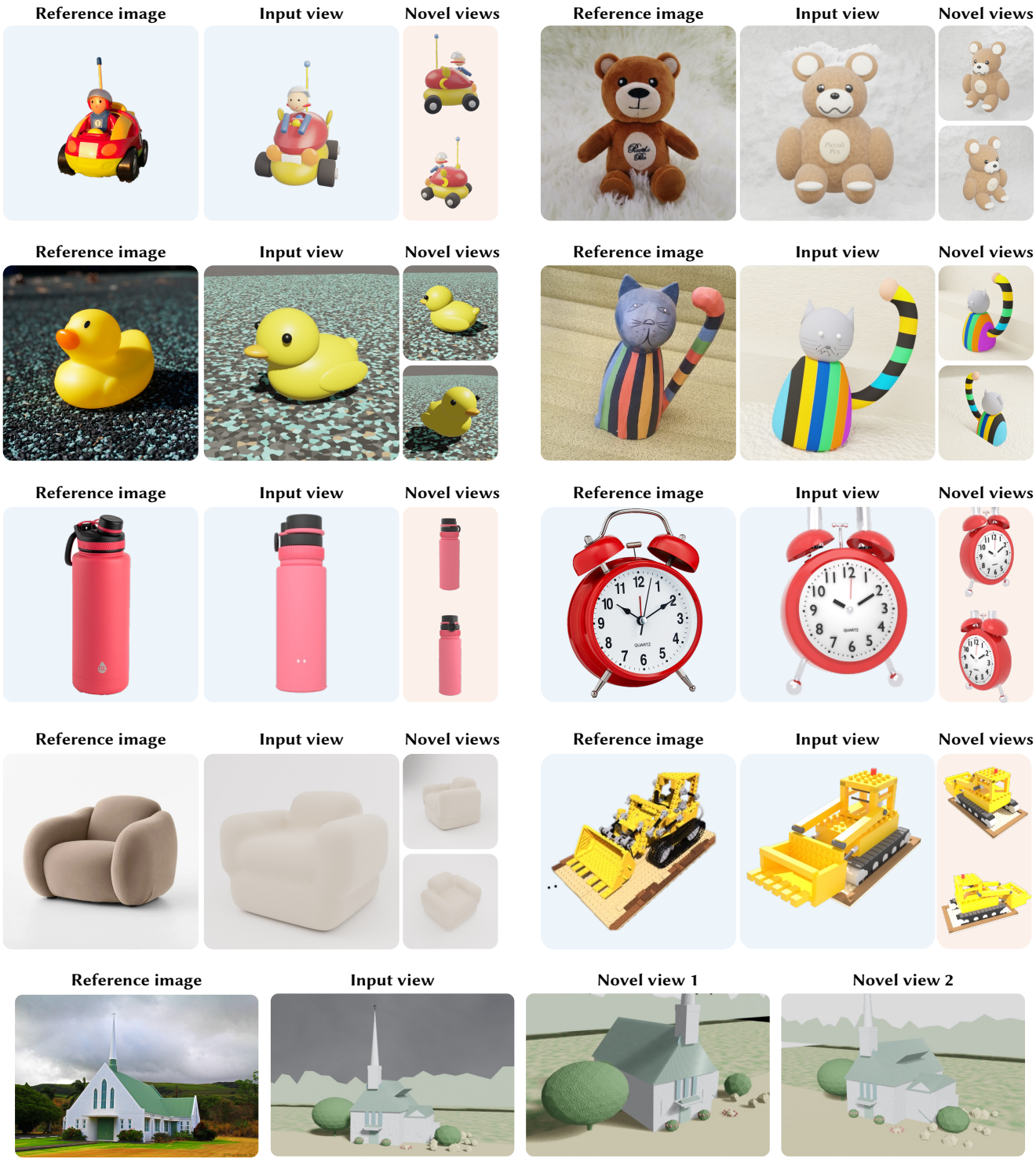


Fig. 8. Gallery of Blender scenes created by SEIG from in-the-wild and synthetic reference images. The synthetic scenes correspond to the examples shown in the first row (left) and the fourth row (right). Input and novel views are visualized along with the input reference image.

## References


Anthropic. 2025. Claude Opus 4.7. https://www.anthropic.com/news/claude-opus-4-7.

Harry Barrow, J Tenenbaum, A Hanson, and E Riseman. 1978. Recovering intrinsic scene characteristics. *Computer Vision Systems* (1978).

Sagie Benaim, Frederik Warburg, Peter Ebert Christensen, and Serge Belongie. 2024. Volumetric disentanglement for 3d scene manipulation. In *IEEE/CVF Winter Conference on Applications of Computer Vision*.

Sai Bi, Zexiang Xu, Kalyan Sunkavalli, David Kriegman, and Ravi Ramamoorthi. 2020. Deep 3d capture: Geometry and reflectance from sparse multi-view images. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Boyuan Chen, Zhuo Xu, Sean Kirmani, Brian Ichter, Danny Driess, Pete Florence, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. 2024. SpatialVLM: Endowing Vision-Language Models with Spatial Reasoning Capabilities. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Bingquan Dai, Li Ray Luo, Qihong Tang, Jie Wang, Xinyu Lian, Hao Xu, Minghan Qin, Xudong Xu, Bo Dai, Haoqian Wang, Zhaoyang Lyu, and Jiangmiao Pang. 2025. MeshCoder: LLM-Powered Structured Mesh Code Generation from Point Clouds. *arXiv preprint arXiv:2508.14879* (2025).

Yue Dong, Guojun Chen, Pieter Peers, Jiawan Zhang, and Xin Tong. 2014. Appearance-from-motion: Recovering spatially varying surface reflectance under unknown lighting. *ACM Transactions on Graphics* (2014).

Stephanie Fu, Netanel Y. Tamir, Shobhita Sundaram, Lucy Chai, Richard Zhang, Tali Dekel, and Phillip Isola. 2023. DreamSim: Learning New Dimensions of Human Visual Similarity using Synthetic Data. In *Advances in Neural Information Processing Systems*.

Ruiqi Gao, Aleksander Holynski, Philipp Henzler, Arthur Brussee, Ricardo Martin-Brualla, Pratul P. Srinivasan, Jonathan T. Barron, and Ben Poole. 2024. CAT3D: Create Anything in 3D with Multi-View Diffusion Models. In *Advances in Neural Information Processing Systems*.

Gemini Team. 2023. Gemini: A Family of Highly Capable Multimodal Models. (2023).

Yunqi Gu, Ian Huang, Jihyeon Je, Guandao Yang, and Leonidas Guibas. 2025. BlenderGym: Benchmarking Foundational Model Systems for Graphics Editing. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Berthold KP Horn. 1970. Shape from shading: A method for obtaining the shape of a smooth opaque object from one view. (1970).

Ziniu Hu, Ahmet Iscen, Aashi Jain, Thomas Kipf, Yisong Yue, David A Ross, Cordelia Schmid, and Alireza Fathi. 2024. SceneCraft: An LLM Agent for Synthesizing 3D Scenes as Blender Code. In *International Conference on Machine Learning*.

Haian Jin, Isabella Liu, Peijia Xu, Xiaoshuai Zhang, Songfang Han, Sai Bi, Xiaowei Zhou, Zexiang Xu, and Hao Su. 2024. TensoIR: Tensorial Inverse Rendering. *arXiv preprint arXiv:2304.12461* (2024).

Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics* (2023).

Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. 2023. Segment Anything. In *IEEE/CVF International Conference on Computer Vision*.

Peter Kulits, Haiwen Feng, Weiyang Liu, Victoria Abrevaya, and Michael J. Black. 2024. Re-Thinking Inverse Graphics With Large Language Models. *arXiv preprint arXiv:2404.15228* (2024).

Long Le, Jason Xie, William Liang, Hung-Ju Wang, Yue Yang, Yecheng Jason Ma, Kyle Vedder, Arjun Krishna, Dinesh Jayaraman, and Eric Eaton. 2024. Articulate-Anything: Automatic Modeling of Articulated Objects via a Vision-Language Foundation Model. *arXiv preprint arXiv:2410.13882* (2024).

Lin Li, Zehuan Huang, Haoran Feng, Gengxiong Zhuang, Rui Chen, Chunchao Guo, and Lu Sheng. 2026. Voxhammer: Training-free precise and coherent 3d editing in native 3d space. In *International Conference on 3D Vision*.

Yang Li and Tatsuya Harada. 2022. Non-rigid Point Cloud Registration with Neural Deformation Pyramid. In *Advances in Neural Information Processing Systems*.

Zhengqin Li, Zexiang Xu, Ravi Ramamoorthi, Kalyan Sunkavalli, and Manmohan Chandraker. 2018. Learning to reconstruct shape and spatially-varying reflectance from a single image. *ACM Transactions on Graphics* (2018).

Zhihao Liang, Qi Zhang, Ying Feng, Ying Shan, and Kui Jia. 2024. GS-IR: 3D Gaussian Splatting for Inverse Rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Dingning Liu, Xiaomeng Dong, Renrui Zhang, Xu Luo, Peng Gao, Xiaoshui Huang, Yongshun Gong, and Zhihui Wang. 2023a. 3DAxiesPrompts: Unleashing the 3D Spatial Task Capabilities of GPT-4V. *arXiv preprint arXiv:2312.09738* (2023).

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023b. Visual Instruction Tuning. In *Advances in Neural Information Processing Systems*.

Parker Liu, Chenxin Li, Zhengxin Li, Yipeng Wu, Wuyang Li, Zhiqin Yang, Zhenyuan Zhang, Yunlong Lin, Sirui Han, and Brandon Y. Feng. 2025. IR3D-Bench: Evaluating Vision-Language Model Scene Understanding as Agentic Inverse Rendering. *arXiv preprint arXiv:2506.23329* (2025).

Sining Lu, Guan Chen, Nam Anh Dinh, Itai Lang, Ari Holtzman, and Rana Hanocka. 2025. LL3M: Large Language 3D Modelers. *arXiv preprint arXiv:2508.08228* (2025).

Rundong Luo, Hong-Xing Yu, and Jiajun Wu. 2025. Unsupervised Discovery of Object-Centric Neural Fields. *Transactions on Machine Learning Research* (2025).

Stephen Robert Marschner. 1998. *Inverse Rendering for Computer Graphics*. Ph. D. Dissertation. Cornell University.

Meta AI Research. 2025. SAM 3D: 3Dfy Anything in Images. *arXiv preprint arXiv:2511.16624* (2025).

Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. 2020. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *European Conference on Computer Vision*.

Tom Monnier, Jake Austin, Angjoo Kanazawa, Alexei Efros, and Mathieu Aubry. 2023. Differentiable blocks world: Qualitative 3d decomposition by rendering primitives. In *Advances in Neural Information Processing Systems*.

Jacob Munkberg, Jon Hasselgren, Tianchang Shen, Jun Gao, Wenzheng Chen, Alex Evans, Thomas Müller, and Sanja Fidler. 2022. Extracting Triangular 3D Models, Materials, and Lighting From Images. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Giljoo Nam, Joo Ho Lee, Diego Gutierrez, and Min H Kim. 2018. Practical svbrdf acquisition of 3d objects with unstructured flash photography. *ACM Transactions on Graphics* (2018).

Felix O'Mahony, Roberto Cipolla, and Ayush Tewari. 2025. VDAWorld: World Modelling via VLM-Directed Abstraction and Simulation. *arXiv preprint arXiv:2512.11061* (2025).

OpenAI. 2023. GPT-4V(ision) System Card. (2023).

Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. 2024. DINOv2: Learning Robust Visual Features without Supervision. *Transactions on Machine Learning Research* (2024).

Ava Pun, Kangle Deng, Ruixuan Liu, Deva Ramanan, Changliu Liu, and Jun-Yan Zhu. 2025. Generating Physically Stable and Buildable Brick Structures from Text. *arXiv preprint arXiv:2505.05469* (2025).

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*.

Ravi Ramamoorthi and Pat Hanrahan. 2001. A Signal-Processing Framework for Inverse Rendering. In *ACM SIGGRAPH*.

Lawrence G Roberts. 1963. *Machine perception of three-dimensional solids*. Ph. D. Dissertation. Massachusetts Institute of Technology.

Gopal Sharma, Rishabh Goyal, Difan Liu, Evangelos Kalogerakis, and Subhransu Maji. 2018. Csgnet: Neural shape parser for constructive solid geometry. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Dor Verbin, Peter Hedman, Ben Mildenhall, Todd Zickler, Jonathan T. Barron, and Pratul P. Srinivasan. 2022. Ref-NeRF: Structured View-Dependent Appearance for Neural Radiance Fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. 2025. VGGT: Visual Geometry Grounded Transformer. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Hanfeng Wu, Xingxing Zuo, Stefan Leutenegger, Or Litany, Konrad Schindler, and Shengyu Huang. 2024. Dynamic lidar re-simulation using compositional neural fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Bangbang Yang, Yinda Zhang, Yinghao Xu, Yijin Li, Han Zhou, Hujun Bao, Guofeng Zhang, and Zhaopeng Cui. 2021. Learning object-compositional neural radiance field for editable scene rendering. In *IEEE/CVF International Conference on Computer Vision*.

Shaofeng Yin, Jiaxin Ge, Zora Zhiruo Wang, Xiuyu Li, Michael J. Black, Trevor Darrell, Angjoo Kanazawa, and Haiwen Feng. 2026. Vision-as-Inverse-Graphics Agent via Interleaved Multimodal Reasoning. *arXiv preprint arXiv:2601.11109* (2026).

Kai Zhang, Fujun Luan, Qianqian Wang, Kavita Bala, and Noah Snavely. 2021a. PhySG: Inverse Rendering with Spherical Gaussians for Physics-based Material Editing and Relighting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Xiuming Zhang, Pratul P Srinivasan, Boyang Deng, Paul Debevec, William T Freeman, and Jonathan T Barron. 2021b. NeRFactor: Neural Factorization of Shape and Reflectance Under an Unknown Illumination. *ACM Transactions on Graphics* (2021).

Yunzhi Zhang, Zizhang Li, Matt Zhou, Shangzhe Wu, and Jiajun Wu. 2024. The Scene Language: Representing Scenes with Programs, Words, and Embeddings. *arXiv preprint arXiv:2410.16770* (2024).